UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

Informatics Institute of Technology

Client-Server Architectures

Student Name – K.K Ranathunga

UoW Student Number - w2120050

IIT Student Number – 20240584

Client-Server Architectures - Coursework (2025/26)

Coursework: Developing a JAX-RS RESTful Service

Part 1: Service Architecture & Setup

1.1 Project & Application Configuration

Question: In your report, explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions.

Answer: JAX-RS by default implements request-based lifecycle, which means that a fresh object of the Resource type (RoomResource) is created each time an HTTP request is received, and once the response is generated, it is garbage collected. This way, it is not possible to use instance variable to persist data. For keeping track of data, the variables need to be static to keep the same information across all the incoming requests in memory. As multiple threads work together in a web server for concurrent requests, there are possibilities of race condition when using normal collection like HashMap.

1.2 The "Discovery" Endpoint

Question: Why is the provision of "Hypermedia" (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

Answer: The Hypermedia As The Engine Of Application State (HATEOAS) principle ensures that REST APIs are self-describing and self-contained. By enabling the API to provide navigational links from the JSON responses, developers do not have to hard-code any URL information from the documentation (which may not remain relevant). The advantage for developers is that even if there are changes in the routing scheme of the API, the client software will automatically adjust itself to the new situation.

Part 2: Room Management

2.1 Room Resource Implementation

Question: When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client side processing.

Answer: Responding only with IDs greatly decreases the data volume to be transferred, minimizing the use of bandwidth while improving response time. On the other hand, it requires the client to issue additional GET requests to receive the real information on each particular room, causing the

N+1 problem and increasing the time needed for processing. Sending back objects consumes more data transfer space, but then the client-side software does not have to issue any additional network queries.

2.2 Room Deletion & Safety Logic

Question: Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple time.

Answer: Absolutely, the DELETE method is idempotent. For a RESTful API, idempotency implies that performing the same actions repeatedly produces the same state on the server as performing the action once would have done. If the DELETE method is used to delete an existing room, then that room gets deleted and a success response is received. If we use the same request again and again, the server will respond with 404 Not Found (as the resource doesn't exist anymore). Thus, the server ends up with the same state regardless of whether the request is performed one or ten times.

Part 3: Sensor Operations & Linking

3.1 Sensor Resource & Integrity

Question: We explicitly use the @Consumes (MediaType.APPLICATION_JSON) annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

Answer: However, in case the payload sent by the user is not supported, meaning the payload's content type differs from what the application expects, the JAX-RS runtime will stop the execution of the request at a point where it never reaches the java method. In such a scenario, a response code of HTTP 415 Unsupported Media Type is thrown back to the user.

3.2 Filtered Retrieval & Search

Question: You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path (e.g., /api/vl/sensors/type/CO2). Why is the query parameter approach generally considered superior for filtering and searching collections?

Answer: According to RESTful design principles, the URL path should define the essence of the resource in question. The concept of filtering is optional; hence, it cannot be an identity but can only be performed on a basic collection. Query strings such as (?type=CO2) are better suited to perform searches because they are optional, order independent, and scalable. In cases where filtering involves

several conditions, such as filtering by type and status, it can easily be achieved using query strings such as (?type=CO2&status=ACTIVE).

Part 4: Deep Nesting with Sub – Resources

4.1 The Sub-Resource Locator Pattern

Question: Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive controller class?

Answer: Sub-Resource Locator Pattern Implements the Single Responsibility Principle. In absence of which, there will be an occurrence where all logic related to management of sensor data, such as validation, filtering, etc., along with the entire cycle of history of sensors will reside within the parent class itself. However, using this approach, we are able to move responsibility for managing the nested path (/{sensorId}/readings) into another new class called SensorReadingResource and keep all logic concerning the Sensor aggregate in the parent class.

Part 5: Advanced Error Handling, Exception Mapping & Logging

5.2 Dependency Validation

Question: Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

Answer: The response code 404 (Not Found) suggests that the endpoint URL in question is incorrect. But in this case, posting sensors involves a completely valid URL (/api/v1/sensors). The HTTP 422 (Unprocessable Entity) response code is appropriate since the server knows the media type of the request entity and its JSON format is correct; however, the server cannot follow the request because it contains semantic errors (i.e., lack of foreign keys for roomId).

5.4 The Global Safety Net (500)

Question: From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

Answer: Showing Java stack traces in their raw form is a serious security threat referred to as "Information Disclosure." The attacker has access to all the inner structure, including directory structures, classes, and methods, that exist inside the backend system. What is even more worrying about this security issue is that stack traces provide a direct indication of the framework's version (e.g., Jersey, Grizzly, etc.). This enables the attacker to refer to the CVE database, where the vulnerability will be precisely targeted without guessing the server's structure.

5.5 API Request & Response Logging Filters

Question: Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting Logger.info() statements inside every single resource method?

Answer: The use of JAX-RS filters allows us to separate concerns from each other. By adding Logger.info () in each of our resource methods, we will end up cluttering the code, breaking the Don't Repeat Yourself principle, as well as forgetting to do so for each endpoint that might be created. The filter provides us with a way of centralizing such a thing.